

📋 Scrap & Sprout — End-to-End 시나리오 문서

버전 v1.1 | 작성일 2026.04 | 작성 김효은

📌 문서 목적

각 시나리오는 플레이어 입력부터 **Unity → Photon → Spring → FastAPI → MySQL → 다시 Unity**까지 전 레이어 흐름을 단계별로 명세합니다. 정상 흐름(Happy Path)과 실패 경로(Failure Path)를 함께 포함합니다.

✅ 확정 설계 결정 사항

항목	결정 내용
Gemini 타임아웃	Unity에서 "NPC가 생각 중..." 즉시 표시 → 5초 대기 → 초과 시 "다시 말을 걸어주세요" 안내. 플레이어가 직접 재시도
Spring DB 실패	즉시 실패 대신 Retry 로직 적용. 일정 횟수 재시도 후 최종 실패 시 안내 팝업
Photon 연결 끊김	<code>OnDisconnectedFromPhoton</code> 자체 콜백 활용 → 재연결 시도 UI 표시
인증 토큰	<code>UnityWebRequest</code> 로 로그인 요청 → 응답값 토큰을 이후 모든 API 헤더에 첨부. (브라우저-백엔드 HTTP 패턴 동일)
타일맵 교체	씨앗 식재 시 Unity가 직접 Photon RPC 호출 → 다른 클라이언트 타일을 새 에셋으로 교체 (Spring 경유 없음)
동시 수거 충돌	<code>PhotonView.RequestOwnership()</code> 사용. 먼저 Ownership 획득한 플레이어가 수거 성공. 늦은 플레이어는 실패 모션 후 IDLE 복귀
인원별 스케일링	자유 선택 + 스탯 보너스 차별화(C+D). 접속 인원에 따라 쓰레기 총량·악천후 강도 자동 조정

📁 포함 시나리오 목록

- S-01 세션 입장 — 로그인 · JWT · 인원별 스케일링
- S-02 쓰레기 수거 — 정상 흐름 + RequestOwnership 충돌 케이스
- S-03 NPC 거래 + Gemini AI 대사 생성 (타임아웃 재시도 포함)
- S-04 씨앗 식재 → Photon RPC 직접 타일 교체 + 맵 단계 전환
- S-05 날씨 이변 발생 → 전체 클라이언트 브로드캐스트
- S-06 환경 연동형 쓰레기 리젠 감소 (F11)

S-01 — 세션 입장

플레이어가 로그인 후 Photon 룸에 입장하고 인원별 스케일링이 완료되기까지의 초기화 흐름입니다. JWT 발급·저장, 세션 초기화, 맵 데이터 수신 포함.

정상 흐름

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
1	플레이어	게임 실행 및 ID/PW 입력	로그인 화면 입력 폼	로그인 요청 준비
2	Unity	<code>UnityWebRequest.Post("/api/auth/login")</code>	HTTP · <code>{ user_name, password }</code>	Spring에 인증 요청
3	Spring	DB 유저 조회 및 비밀번호 검증	<code>SELECT * FROM USER WHERE user_name=?</code>	일치 시 JWT 생성
4	Spring	JWT 발급 및 응답	HTTP 200 · <code>{ token, user_id, user_name }</code>	Unity에 토큰 전달
5	Unity	토큰 수령 및 저장	<code>PlayerPrefs.SetString("jwt", token)</code>	이후 모든 <code>UnityWebRequest</code>
6	Unity	Photon 룸 생성 또는 참여	<code>PhotonNetwork.JoinOrCreateRoom()</code> · playerCount 확정	Photon 세션 연결
7	Unity	<code>UnityWebRequest.Post("/api/session/init")</code>	HTTP · <code>{ playerCount, mapId }</code> · JWT 헤더	Spring에 세션 초기화 요청
8	Spring	<code>player_count_modifier</code> 조회	<code>SELECT modifier FROM MAPS WHERE map_id=?</code>	인원 수 배율 계산
9	Spring	쓰레기 총량 계산 및 응답	<code>trash_total = base × modifier</code> · HTTP 200	스케일링 완료 데이터 반환
10	Unity	맵 오브젝트 생성, 게임 시작	Photon <code>NetworkObject</code> 스폰	플레이어 조작 가능

실패 경로

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
E1	Spring	비밀번호 불일치	HTTP 401 Unauthorized	Unity: 로그인 실패 팝업 표시
E2	Spring	DB 조회 실패	HTTP 500	Unity: "서버 오류" 안내 팝업
E3	Photon	룸 연결 끊김	<code>OnDisconnectedFromPhoton</code> 콜백 자동 호출	Unity: 재연결 시도 UI 표시 → 자동 재접속 시도

S-02 — 쓰레기 수거

플레이어가 E키로 쓰레기를 수거할 때 `PhotonView.RequestOwnership()` 으로 선점 판정하고 Spring 인벤토리에 저장하는 흐름입니다. 2인 동시 수거 충돌 케이스 포함.

정상 흐름

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
1	플레이어	쓰레기 접근 후 E키 입력	Trigger Collider 감지	수거 시도 시작
2	Unity	<code>trashPhotonView.RequestOwnership()</code> 호출	Photon UDP · Ownership 요청	Photon이 선점 판정
3	Photon	Ownership 부여 → <code>OnOwnershipTransferred</code> 콜백	먼저 요청한 플레이어에게 Ownership 할당	해당 플레이어 수거 권
4	Unity	<code>UnityWebRequest.Post("/api/inventory/add")</code>	HTTP · <code>{ userId, trashId, qty:1 }</code> · JWT 헤더	Spring에 인벤토리 저
5	Spring	유저 검증 및 인벤토리 INSERT	<code>@Transactional</code> · <code>INSERT INTO INVENTORY ...</code>	DB 저장 완료
6	Spring	성공 응답	HTTP 200 · <code>{ inventoryId, quantity }</code>	Unity에 결과 전달
7	Unity	Photon RPC로 수거 완료 브로드캐스트	UDP · <code>Rpc_CollectTrash(trashId)</code> → All Clients	전체 화면 오브젝트 비
8	Unity	HUD 인벤토리 +1 업데이트	<code>UIManager.UpdateInventory()</code>	수거 완료

동시 수거 충돌 (2인이 동시에 E키)

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
1	A·B 동시	동시에 E키 → <code>RequestOwnership()</code> 동시 호출	두 요청이 Photon 서버에 거의 동시 도달	Photon이 먼저 도착한 요청 선택
2	Photon	A에게 Ownership 부여	B의 요청 묵시적 거부 (Ownership 미할당)	A만 <code>OnOwnershipTransferred</code> 수신
3	Unity(B)	Ownership 미수신 감지	<code>OnOwnershipTransferred</code> 콜백 미호출 확인	실패 모션(애니메이션) 재생 후 IDLE/WALK 복귀
4	A	3~8단계 정상 흐름 동일	—	A만 인벤토리 +1

실패 경로

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
E1	Spring	DB INSERT 실패	HTTP 500 · <code>@Transactional</code> 자동 롤백	Retry 로직 실행 (최대 N회 재시도) → 최종 실패 시 팝업 + Ownership 해제
E2	Photon	수거 중 연결 끊김	<code>OnDisconnectedFromPhoton</code> 콜백	재연결 시도 UI 표시 → 재접속 후 인벤토리 서버 재조회

S-03 — NPC 거래 및 Gemini AI 대사 생성

NPC 상점에서 거래 시 Spring이 검증하고 FastAPI가 Gemini를 비동기 호출합니다.

Gemini 응답 대기 중 Unity에서 "NPC가 생각 중..." 문구를 표시하고, 5초 초과 시 재시도 안내로 전환합니다.

정상 흐름 — 쓰레기 판매

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
1	플레이어	NPC 접근 후 E키 → 거래 UI 오픈	Trigger Collider 감지	거래 UI 활성화
2	Unity	<code>UnityWebRequest.Post("/api/npc/interact")</code>	HTTP · <code>{ userId, npcId, itemNo, qty }</code> · JWT 헤더	Spring에 상호작용 요청
2-1	Unity	"NPC가 생각 중..." 로딩 문구 즉시 표시	<code>UIManager.ShowThinking()</code>	AI 응답 대기 중임을 플레이어에게 알림
3	Spring	유저 Gold·인벤토리 조회	<code>SELECT gold, qty FROM USER, INVENTORY ...</code>	보유 상태 확인
4	Spring	<code>POST http://ai-server/npc/dialogue</code>	HTTP · <code>{ weather, pollutionLv, gold, itemName, ... }</code>	FastAPI에 대사 생성 요청
5	FastAPI	<code>async def</code> → Gemini API <code>await</code> 호출	HTTPS · 프롬프트 JSON → Gemini API	Gemini 응답 대기 (비동기)
6	Gemini	대사 및 가격 배율 생성	<code>{ dialogue: "...", price_multiplier: 1.2 }</code>	AI 응답 반환
7	FastAPI	<code>NPC_INTERACTIONS</code> INSERT 후 반환	<code>await db.commit()</code> · HTTP 200 → Spring	대사 로그 DB 저장
8	Spring	거래 검증 및 Gold 지급	<code>@Transactional</code> · <code>gold += price × multiplier</code>	재화 트랜잭션 완료
9	Spring	최종 응답	HTTP 200 · <code>{ dialogue, newGold, priceMultiplier }</code>	Unity에 결과 전달
10	Unity	"NPC가 생각 중..." 제거 → AI 대화창 출력 + Gold HUD 갱신	<code>UIManager.HideThinking()</code> · <code>ShowDialogue()</code>	거래 완료

실패 경로

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
E1-1	Unity	요청 후 5초 경과, 응답 없음	<code>Coroutine</code> 타이머 5초 만료	"다시 말을 걸어주세요" 안내 문구 표시 → 거래 UI 닫힘. 플레이어 재
E1-2	FastAPI	Gemini 5초 초과	<code>asyncio.wait_for</code> TimeoutError	Spring에 timeout 에러 반환
E2	Spring	Gold 부족 검증 실패	HTTP 400 · <code>{ error: "INSUFFICIENT_GOLD" }</code>	Unity: "골드가 부족합니다" 팝업. 거래 취소
E3	Spring	DB 거래 실패	HTTP 500 · <code>@Transactional</code> 롤백	Retry 로직 실행 → 최종 실패 시 "거래 오류" 팝업. 변화 없음

S-04 — 씨앗 식재 및 타일맵 실시간 교체

씨앗 식재 시 Unity가 직접 Photon RPC로 다른 클라이언트 타일을 교체합니다 (Spring 경우 없음).

`pollution_level` 이 임계값을 넘으면 맵 전체 단계 전환이 추가 발생합니다.

맵 단계 전환 임계값

단계	pollution_level	타일 에셋	게임플레이 변화
1단계 (오염)	70 ~ 100	회색 오염 타일	산성비 빈발 · 시야 제한 · 쓰레기 리젠 MAX
2단계 (정화 진행)	30 ~ 70	황토색 타일 · 새싹	날씨 안정화 · 리젠 50% 감소 · 식물 성장 시작
3단계 (복원 완료)	0 ~ 30	초록 타일 · 꽃 나무	맑은 날씨 고정 · 리젠 거의 없음 · 배터리 소모 감소

정상 흐름 — 씨앗 식재 + 개별 타일 교체 (매 식재 시)

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
1	플레이어	씨앗 선택 후 오염 타일에 F키	인벤토리 씨앗 수량 확인	식재 위치 확정
2	Unity	<code>UnityWebRequest.Post("/api/map/plant")</code>	HTTP · <code>{ userId, mapId, tileX, tileY }</code> · JWT 헤더	Spring에 식재 요청
3	Spring	씨앗 인벤토리 차감	<code>@Transactional</code> · <code>UPDATE INVENTORY qty-=1</code>	씨앗 소비
4	Spring	<code>pollution_level</code> 감소 + <code>tree_count+1</code>	<code>UPDATE MAPS SET pollution_level-=2.5, tree_count+=1</code>	DB 환경 데이터 갱신
5	Spring	HTTP 200 응답	<code>{ newPollutionLevel, treeCount }</code>	Unity에 결과 전달
6	Unity	Photon RPC 직접 호출	UDP · <code>Rpc_PlantSeed(tileX, tileY, newAssetId)</code> → All Clients	다른 클라이언트에 E
7	Unity(전체)	해당 타일 에셋 교체	<code>TileMap.SetTile(pos, newSprite)</code>	오염 타일 → 정화 타
8	Spring	쓰레기 리젠 파라미터 재계산 (비동기)	<code>regen_rate = base × (pollution_level / 100)</code>	리젠 빈도·총량 축소

맵 단계 전환 흐름 (임계값 돌파 시 추가 발생)

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
T1	Spring	<code>pollution_level</code> 임계값 돌파 감지	예: 72.0 → 69.5 (70 미만 진입)	2단계 전환 트리거
T2	Spring	전체 맵 단계 변경 응답에 포함	HTTP 200 · <code>{ newPollutionLevel, newStage: 2 }</code>	Unity가 단계 전환 감지
T3	Unity	단계 전환 감지 → Photon RPC 전체 맵 교체	UDP · <code>Rpc_MapStageChange(stage:2, tileSetId)</code> → All	전체 클라이언트에 맵 교체
T4	Unity(전체)	전체 타일맵 에셋 일괄 교체	<code>TileMap.SwapTileSet(stage2_tileset)</code>	회색 → 황토색 타일
T5	Unity(전체)	URP Post-Processing 변경	Color Grading · Fog 밀도 감소	화면 전체 분위기 전환
T6	Unity(전체)	날씨 확률 파라미터 갱신	산성비 40% → 20%로 감소	F08 환경 복원 연동
T7	Unity(전체)	HUD 정화 진행률 업데이트	<code>PollutionBar</code> 수치 갱신 · 단계 아이콘 변경	플레이어에게 달성률 표시

실패 경로

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
E1	Spring	씨앗 수량 0	HTTP 400 · <code>{ error: "NO_SEED" }</code>	Unity: "씨앗이 없습니다" 팝업. Photon RPC 호출 안 함
E2	Photon	RPC 일부 클라이언트 미수신	네트워크 패킷 손실	재접속 시 <code>GET /api/session/state</code> 로 현재 stage 재동기화
E3	Spring	DB 갱신 실패	HTTP 500 · <code>@Transactional</code> 롤백	Retry 로직 실행. 최종 실패 시 Photon RPC 호출 안 함 → 타일 변경 지연

S-05 — 날씨 이변 발생 및 브로드캐스트

Spring 서버 타이머(`@Scheduled`)와 `pollution_level` 기반 확률 알고리즘이 날씨를 결정하고 Photon을 통해 전체 클라이언트에 동시 적용합니다.

정상 흐름

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
1	Spring	날씨 타이머 만료 (기본 5분)	<code>@Scheduled</code> 어노테이션 서버 타이머	날씨 결정 로직 실행
2	Spring	<code>pollution_level</code> 기반 확률 계산	$level \geq 70 \rightarrow$ 산성비 40% · 황사 30% · 맑음 30%	날씨 종류 결정
3	Spring	<code>ENVIRONMENTS</code> 테이블 패널리티 조회	<code>SELECT * FROM ENVIRONMENTS WHERE weather_name=?</code>	날씨 효과 데이터 로드
4	Spring	Photon HTTP API로 날씨 브로드캐스트	POST Photon API · <code>{ environment_id, weatherName, penalty }</code>	전체 클라이언트에 날씨 전파
5	Photon	룸 전체 날씨 RPC 브로드캐스트	UDP · <code>OnWeatherChange(envId, weatherName)</code> → All	모든 클라이언트 동시 수신
6	Unity(전체)	날씨 비주얼 효과 적용	URP Post-Processing · Fog / Rain Particle 활성화	화면 시각 연출 변경
7	Unity(전체)	날씨 패널리티 적용	<code>WeatherMultiplier</code> 갱신 → 배터리 소모율 1.5배 등	게임플레이 수치 변경
8	Unity(전체)	HUD 날씨 아이콘 + 경고 메시지 출력	<code>UIManager.ShowWeatherAlert(weatherName)</code>	플레이어 날씨 변화 인지

실패 경로

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
E1	Photon	일부 클라이언트 RPC 미수신	네트워크 패킷 손실	<code>OnDisconnectedFromPhoton</code> → 재접속 후 <code>GET /api/session/state</code> 로 날씨 동기화
E2	Spring	타이머 과부하 지연	<code>@Scheduled</code> 지연 실행	최대 30초 지연 허용. 날씨는 비크리티컬

S-06 — 환경 연동형 쓰레기 리젠 감소 (F11)

`pollution_level` 이 낮아질수록 쓰레기 리젠 총량과 빈도가 실시간으로 줄어드는 피드백 루프입니다.

정상 흐름

단계	주체	행동	데이터 / 프로토콜	결과 / 상태
1	Spring	S-02 또는 S-04 완료 후 <code>pollution_level</code> 갱신	최신 <code>pollution_level</code> 값 확인	리젠 재계산 트리거
2	Spring	리젠 파라미터 재계산	<code>regen_rate = base_rate × (level/100)</code>	현재 오염도 비례 계산
3	Spring	갱신된 파라미터 DB 저장	<code>UPDATE MAPS SET regen_rate=?, max_trash_count=?</code>	DB 반영
4	Spring	Photon HTTP API로 파라미터 브로드캐스트	<code>POST Photon API · { regenRate, maxTrashCount }</code>	전체 클라이언트 파라미터화
5	Photon	리젠 파라미터 RPC 브로드캐스트	UDP · <code>OnRegenParamChange(rate, max)</code> → All	모든 클라이언트 수신
6	Unity(전체)	<code>TrashSpawner</code> 파라미터 갱신	<code>_spawnInterval = 1f / regenRate</code>	리젠 간격: 최대 수 감소
7	Unity(전체)	HUD 리젠 인디케이터 갱신	<code>RegenRateBar</code> 시각화 업데이트	플레이어에게 리젠 감도 표시

리젠 계산 예시

pollution_level	regen_rate	리젠 간격	max_count
100 (최초)	1.0	300초마다	100개
70 (1단계 경계)	0.7	약 428초마다	70개
30 (2→3단계)	0.3	약 1,000초마다	30개
0 (완전 복원)	0.0	리젠 없음	0개

부록 A — 핵심 REST API 엔드포인트

시나리오	메서드·경로	요청 주체	핵심 Body	응답 핵심 필드
S-01	<code>POST /api/auth/login</code>	Unity → Spring	<code>user_name, password</code>	<code>token, user_id</code>
S-01	<code>POST /api/session/init</code>	Unity → Spring	<code>playerCount, mapId</code>	<code>trash_total, modifier</code>
S-02	<code>POST /api/inventory/add</code>	Unity → Spring	<code>userId, trashId, quantity</code>	<code>inventoryId, quantity</code>
S-03	<code>POST /api/npc/interact</code>	Unity → Spring	<code>userId, npcId, itemNo, quantity</code>	<code>dialogue, newGold, priceMultiplier</code>
S-03	<code>POST /npc/dialogue</code>	Spring → FastAPI	<code>weather, pollutionLevel, gold, item...</code>	<code>dialogue, price_multiplier</code>
S-04	<code>POST /api/map/plant</code>	Unity → Spring	<code>userId, mapId, tileX, tileY</code>	<code>newPollutionLevel, treeCount, newStage</code>
S-05 / 재접속	<code>GET /api/session/state</code>	Unity → Spring	<code>sessionId</code>	<code>weatherName, pollutionLevel, currentStage</code>

부록 B — Photon RPC 목록